4.1 Authentication Test Cases (test_auth.py)

API Tested	Inputs	Expected Output	Actual Output	Result
POST /auth/register	Valid payload (full_name, valid email, password >= 8 chars, matching confirmation)	HTTP 201, success message, and access_token returned	HTTP 201, token generated and persisted	**PASS**
POST /auth/register	Mismatched password and confirm_password fields	HTTP 422 Unprocessable Entity (Pydantic validation error)	HTTP 422 returned	**PASS**
POST /auth/register	Duplicate email address registration attempt	HTTP 400 Bad Request, "Email is already registered."	HTTP 400 returned with correct detail	**PASS**
POST /auth/login	Valid registered credentials via OAuth2 form data	HTTP 200 OK, access_token and bearer token type returned	HTTP 200, valid token issued	**PASS**
POST /auth/login	Non-existent email or incorrect password	HTTP 401 Unauthorized, "Incorrect email or password"	HTTP 401 returned	**PASS**

POST /auth/logout	Valid Bearer token passed in the Authorization header	HTTP 200 OK, logout confirmation message	HTTP 200 returned	

3.2 Users Module (/users)

Method	Endpoint	Roles	User Story	Description
GET	/users/me	Authenticated	Profile Management	Returns the current authenticated resident's account details and profile attributes.
PATCH	/users/me/onboarding	Authenticated	Profile Onboarding	Saves the 3-step user onboarding survey (interests, skills offered, volunteering interest) to the database.

GET	/users/me/dashboard	Authenticated	Resident Hub Dashboard	Aggregates and returns core metrics including wallet balance, active service listings, and volunteering event counts.
PATCH	/users/me/settings	Authenticated	Account Settings	Updates the resident's basic account profile details (full name and email).

3. Test Cases Table Entry (test_users.py)

Add these rows to section **4.2** of your project report table:

API Tested	Inputs	Expected Output	Actual Output	Result
GET /users/me	Valid Bearer token / Mocked current user	HTTP 200 OK, resident profile object returned	HTTP 200, profile data returned successfully	**PASS**
PATCH /users/me/onboarding	Valid survey payload (interests list, skills_offered, volunteer_interest)	HTTP 200 OK, updated resident profile with comma-sepa	HTTP 200, fields persisted correctly	**PASS**

		rated interests		
GET /users/me/dashboard	Authenticated resident session	HTTP 200 OK, aggregated hub dashboard stats and credit balances	HTTP 200, stats rollups computed	**PASS**
PATCH /users/me/settings	Updated full_name and email payload	HTTP 200 OK, confirmation message and updated user object	HTTP 200, record successfully updated	**PASS**

4.2 Users & Onboarding Module Test Cases (test_users.py)

API Tested	Inputs	Expected Output	Actual Output	Result
GET /users/me	Valid authentication credentials / mock resident session	HTTP 200 OK, resident profile details and email	HTTP 200, correct profile data returned	**PASS**
PATCH /users/me/onboarding	Valid survey payload (interests list, skills_offered, volunteer_interest)	HTTP 200 OK, updated resident profile with comma-sepa	HTTP 200, survey responses saved successfully	**PASS**

		rated interests		
GET /users/me/dashboard	Authenticated resident session	HTTP 200 OK, aggregated hub stats including wallet balance and active services	HTTP 200, metrics computed and returned	**PASS**
PATCH /users/me/settings	Updated full_name and email payload	HTTP 200 OK, success confirmation message and updated user object	HTTP 200, settings successfully updated	**PASS**

3.3 Societies & Onboarding Module (/societies)

Method	Endpoint	Roles	User Story	Description
GET	/societies/search	Public / Auth	Society Lookup	Searches and verifies a society using its unique invite key.

GET	/societies/{society_id}/households	Public / Auth	Unit Selection	Fetches the list of all households and their occupation states (is_taken) for a given society.
POST	/societies/join	Authenticated	Member Onboarding	Submits a join request with a flat number and uploads an ID proof document directly to cloud storage.
GET	/societies/join/status	Authenticated	Request Status Tracking	Retrieves the latest status of the resident's pending or approved society join request.

POST	/societies/apply	Authenticated	New Society Proposal	Submits an application to register a brand-new residential community on the platform.
GET	/societies/admin/join-requests	Admin / Owner / Coordinator	Membership Moderation	Fetches pending membership requests and linked ID proofs for approval review.
POST	/societies/{society_id}/households/bulk	Admin / Owner	Registry Setup	Dynamically generates a sequential range of household units (blocks and flats) for a community.

3. Test Cases Table Entry (test_societies.py)

Add these rows to section **4.3** of your project report table:

API Tested	Inputs	Expected Output	Actual Output	Result
GET /societies/search	Valid invite_key parameter query	HTTP 200 OK, society basic profile information returned	HTTP 200, matching society returned	**PASS**
GET /societies/search	Invalid or nonexistent invite_key query	HTTP 404 Not Found error message	HTTP 404 returned properly	**PASS**
GET /societies/{id}/households	Valid society_id route parameter	HTTP 200 OK, list of house units and occupancy flags	HTTP 200, household array returned	**PASS**
POST /societies/apply	Valid application details (society_name, address, count, reason)	HTTP 201 Created, owner review confirmation message	HTTP 201, proposal successfully logged	**PASS**
POST /societies/{id}/households/bulk	Admin credentials and unit range configuration payload	HTTP 201 Created, batch generation summary response	HTTP 201, units created successfully	**PASS**

3.4 Wallet Module (/wallet)

Method	Endpoint	Roles	User Story	Description
GET	/wallet	Authenticated	Credit Economy Management	Retrieves the current resident's token balance and chronological history of credit transactions (earned/spent).

4.4 Wallet Module Test Cases (test_wallet.py)

API Tested	Inputs	Expected Output	Actual Output	Result
GET /wallet/	Authenticated resident session (no existing wallet record)	HTTP 200 OK, initializes default balance (25 credits) and empty transaction ledger	HTTP 200, default wallet initialized and returned	**PASS**
GET /wallet/	Authenticated resident session (pre-existing wallet and transaction history)	HTTP 200 OK, correct custom balance and chronologically sorted transaction ledger	HTTP 200, correct balance and transaction list returned	**PASS**

3.5 Marketplace Module (/marketplace)

Method	Endpoint	Roles	User Story	Description

POST	/marketplace/skills	Authenticated	Skill Listing Submission	Submits a new skill session offer (defaults to PENDING approval status).
POST	/marketplace/resources	Authenticated	Resource Listing Submission	Submits a community resource for sharing (defaults to PENDING approval status).
GET	/marketplace/skills	Authenticated	Browse Skills	Fetches all skill sessions that have been approved by a coordinator (APPROVE).
GET	/marketplace/resources	Authenticated	Browse Resources	Fetches all available and approved community physical resources

				for borrowing.
POST	/marketplace/skills/{id}/book	Authenticated	Book a Skill Session	Books an approved skill, transferring credits atomically from buyer to provider and logging the ledger.
POST	/marketplace/resources/{id}/borrow	Authenticated	Borrow a Resource	Borrows an approved resource, transferring credits atomically, marking the resource unavailable, and recording transactions.

4.5 Marketplace Module Test Cases (test_marketplace.py)

API Tested	Inputs	Expected Output	Actual Output	Result

POST /marketplace/skills	Valid skill payload (title, description, category, tags, hourly rate)	HTTP 200 OK, returns created skill listing with PENDING approval status	HTTP 200, skill successfully created	**PASS**
GET /marketplace/skills	Authenticated resident session query	HTTP 200 OK, returns filtered list containing only coordinator-approved (APPROVE) skill sessions	HTTP 200, approved skills list returned	**PASS**
POST /marketplace/skills/{id}/book	Valid approved skill ID with buyer having sufficient credit balance	HTTP 200 OK, atomic credit transfer executed between buyer and provider, transaction logged	HTTP 200, skill booked successfully	**PASS**
POST /marketplace/resources/{id}/borrow	Valid available and approved resource ID with sufficient balance	HTTP 200 OK, atomic credit transfer executed, resource marked unavailable (is_taken/is_available = False)	HTTP 200, resource successfully borrowed	**PASS**

3.6 Events Module (/events)

Method	Endpoint	Roles	User Story	Description
GET	/events	Authenticated	Community Events Directory	Fetches all upcoming community events along with active volunteer headcounts and the current user's registration status.
POST	/events/{id}/volunteer	Authenticated	Volunteer Request	Submits a self-nomination request for an event, setting status to PENDING_COORDINATOR.
POST	/events/{id}/withdraw	Authenticated	Event Withdrawal	Allows a resident to withdraw from an event roster with a specified reason.
PATCH	/events/{id}/invites/respond	Authenticated	Invite Response	Allows a resident to accept or decline a direct event participation invite (PENDING_RESIDENT).

4.6 Events Module Test Cases (test_events.py)

API Tested	Inputs	Expected Output	Actual Output	Result

GET /events/	Authenticated resident session	HTTP 200 OK, returns upcoming community events list with active volunteer headcounts and user participation status	HTTP 200, events directory and counters returned	**PASS**
POST /events/{id}/volunteer	Valid upcoming event ID	HTTP 200 OK, submits self-nomination and records PENDING_COORDINATOR status	HTTP 200, volunteer request successfully logged	**PASS**
POST /events/{id}/withdraw	Valid event ID and reason_for_leaving payload	HTTP 200 OK, updates status to WITHDRAWN and saves withdrawal reason	HTTP 200, withdrawal processed successfully	**PASS**
PATCH /events/{id}/invites/respond	Valid event ID and invite action payload (ACCEPT / DECLINE)	HTTP 200 OK, updates direct participation invite (PENDING_RESIDENT) to approved/declined status	HTTP 200, invite response saved successfully	**PASS**

3.7 Notices Module (/notices)

Method	Endpoint	Roles	User Story	Description

POST	/notices/	Authenticated	Broadcast Announcement	Creates and broadcasts a new community notice authored by the current user.
GET	/notices/	Authenticated	View Notices Feed	Fetches all community notices sorted chronologically by newest, including author names and system roles for UI tagging.
DELETE	/notices/{id}	Authenticated (Author / Admin / Owner)	Remove Notice	Deletes a specific notice, enforcing that only the original author or platform administrators have deletion privileges.

3. Test Cases Table Entry (test_notices.py)

Add these rows to section 4.7 of your project report table:

API Tested	Inputs	Expected Output	Actual Output	Result
POST /notices/	**Valid notice payload (title, content)**	**HTTP 201 Created, broadcasts the**	**HTTP 201, notice broadcasted**	**PASS**

		announcement successfully	confirmation returned	
GET /notices/	Authenticated resident session	HTTP 200 OK, returns chronological feed of notices with author names and roles	HTTP 200, notices feed returned properly	PASS
DELETE /notices/{id}	Valid notice ID belonging to the authenticated author	HTTP 200 OK, deletes the notice record from the database	HTTP 200, notice deleted successfully	PASS

3.8 Complaints Module (/complaints)

Method	Endpoint	Roles	User Story	Description
POST	/complaints/	Authenticated	File Community Complaint	Submits a new standard grievance or issue report, with support for anonymity.
POST	/complaints/audit	Authenticated	Submit Confidential Audit	Files a confidential performance or conduct audit against a specific coordinator (target_coordinator_id) intended for super administrator review.

GET	/complaints/	Authenticated	View My Complaints	Retrieves all complaints and conduct audits filed by the currently authenticated resident.

3. Test Cases Table Entry (test_complaints.py)

Add these rows to section 4.8 of your project report table:

API Tested	Inputs	Expected Output	Actual Output	Result
POST /complaints/	Valid complaint payload (subject, category, description, is_anonymous)	HTTP 201 Created, registers standard complaint with OPEN status	HTTP 201, complaint successfully filed	PASS
POST /complaints/audit	Valid audit payload targeting a coordinator ID	HTTP 201 Created, registers confidential conduct audit flag (is_audit=True)	HTTP 201, audit sent successfully	PASS
GET /complaints/	Authenticated resident session	HTTP 200 OK, returns list of complaints/audits filed by the calling user	HTTP 200, personal complaints list returned	PASS

3.9 Coordinator Console (/coordinator)

Method	Endpoint	Roles	User Story	Description
GET	/coordinator/dashboard	Coordinator, Admin, Owner	Action Board Metrics	Fetches top-level aggregated counts for pending approvals, open complaints, and upcoming events.
GET	/coordinator/approvals/skills	Coordinator, Admin, Owner	Review Skill Listings	Lists all skill session offers pending coordinator approval (PENDING).
GET	/coordinator/approvals/resources	Coordinator, Admin, Owner	Review Resource Listings	Lists all physical resources pending coordinator approval.
PATCH	/coordinator/approvals/skills/{id}	Coordinator, Admin, Owner	Approve/Reject Skill	Updates skill approval status, enforcing strict self-approval prevention rules.
PATCH	/coordinator/approvals/resources/{id}	Coordinator,	Approve/Reject Resource	Updates physical resource approval status,

		Admin, Owner		preventing self-approval.
GET	/coordinator/complaints	Coordinator, Admin, Owner	Mediation Dashboard	Fetches open community complaints for dispute mediation.
PATCH	/coordinator/complaints/{id}/mediate	Coordinator, Admin, Owner	Mediate Dispute	Saves mediation notes and optionally marks a complaint as resolved.
POST	/coordinator/events	Coordinator, Admin, Owner	Schedule Event	Publishes a new community event with capacity limits and credit rewards.
GET	/coordinator/events/rosters	Coordinator, Admin, Owner	Event Rosters	Retrieves all events along with their complete volunteer sign-up rosters.
PATCH	/coordinator/events/volunteers/{id}/review	Coordinator, Admin, Owner	Review Volunteer / Reward	Approves or rejects volunteer requests, or completes participation to trigger wallet credit rewards.

POST	/coordinator/events/{id}/invite	Coordinator, Admin, Owner	Direct Resident Invite	Sends a direct participation invitation (PENDING_RESIDENT) to a specific resident.

4.10 Coordinator Console Test Cases (test_coordinator.py)

API Tested	Inputs	Expected Output	Actual Output	Result
GET /coordinator/dashboard	Authorized coordinator session	HTTP 200 OK, returns aggregated action board metrics (pending approvals count, open complaints count, upcoming events count)	HTTP 200, metrics summary returned	PASS
PATCH /coordinator/approvals/skills/{id}	Valid skill ID and approval review payload (APPROVE / REJECT)	HTTP 200 OK, updates listing status while blocking self-approval	HTTP 200, skill listing review processed	PASS

PATCH /coordinator/complaints/{id}/mediate	Valid complaint ID and mediation notes payload	HTTP 200 OK, appends mediation log notes and optionally marks grievance as resolved	HTTP 200, mediation notes saved	PASS
POST /coordinator/events	Valid event creation payload including credit_reward and max_volunteers	HTTP 201 Created, schedules a new community event successfully	HTTP 201, event scheduled	PASS
PATCH /coordinator/events/volunteers/{id}/review	Valid volunteer record ID and review action payload	HTTP 200 OK, updates volunteer participation status and manages roster workflows	HTTP 200, status updated successfully	PASS

3.10 Admin Console & Governance (/admin)

Method	Endpoint	Roles	User Story	Description

GET	**/admin/dashboard**	**Admin, Owner**	**Admin Health Dashboard**	**Fetches society capacity metrics, household claim rates, active coordinator counts, and invite keys.**
GET	**/admin/households**	**Admin, Owner**	**View Household Registry**	**Lists all registered housing units sorted by flat number.**
POST	**/admin/households/bulk**	**Admin, Owner**	**Bulk Generate Units**	**Automatically generates a sequential range of housing units (e.g., A-101 to A-105) using pattern wildcards.**
DELETE	**/admin/households/{id}**	**Admin, Owner**	**Remove Unclaimed Unit**	**Deletes a household card from the registry if it has not been claimed by a resident.**

GET	**/admin/residents**	**Admin, Owner**	**Residents Directory**	**Fetches a consolidated roster combining resident profile details, assigned households, and credit balances.**
PATCH	**/admin/coordinators/promote**	**Admin, Owner**	**Promote to Coordinator**	**Elevates a standard resident's system role to Coordinator.**
PATCH	**/admin/coordinators/demote**	**Admin, Owner**	**Strip Coordinator Role**	**Revokes coordinator privileges, returning the user to standard resident status.**
GET	**/admin/audits**	**Admin, Owner**	**Confidential Audits Roster**	**Fetches confidential conduct audit reports filed against specific coordinators.**

PATCH	**/admin/audits/{id}/review**	**Admin, Owner**	**Review Conduct Audit**	**Resolves, warns, or dismisses confidential coordinator audits with administrative audit notes.**
GET	**/admin/metrics/**	**Admin, Owner**	**Performance Analytics**	**Computes circulating credit totals, participation trends, and dispute ratio metrics.**
GET	**/admin/settings/**	**Admin, Owner**	**Admin Profile Settings**	**Retrieves administrator profile and assigned society metadata.**

3. Test Cases Table Entry (test_admin.py)

Add these rows to section 4.11 of your project report table:

API Tested	Inputs	Expected Output	Actual Output	Result

GET /admin/dashboard	Admin/Owner session	HTTP 200 OK, returns society capacity stats, claimed counts, and network metadata	HTTP 200, admin dashboard payload returned	PASS
POST /admin/households/bulk	Bulk generation payload (block_name, address_format, range)	HTTP 201 Created, generates sequential housing units successfully	HTTP 201, units successfully created	PASS
PATCH /admin/coordinators/promote	Valid resident ID payload	HTTP 200 OK, elevates resident system role to Coordinator	HTTP 200, user promoted successfully	PASS
GET /admin/audits	Admin/Owner session	HTTP 200 OK, fetches open confidential conduct audits against	HTTP 200, confidential audits list returned	PASS

		coordinators		
PATCH /admin/audits/{id}/review	Audit ID and review action payload (WARN/RESOLVE/DISMISS)	HTTP 200 OK, resolves audit status and logs administrative review notes	HTTP 200, audit review processed	PASS
GET /admin/metrics/	Admin/Owner session	HTTP 200 OK, calculates platform KPIs, circulating credit sums, and trends	HTTP 200, performance metrics returned	PASS

3.11 System Manager / Owner Console (/owner)

Method	Endpoint	Roles	User Story	Description

GET	**/owner/dashboard**	**Owner**	**Ecosystem Action Board**	**Fetches global platform metrics including active societies, total residents, pending applications, and recent join requests.**
GET	**/owner/analytics**	**Owner**	**Platform Analytics**	**Computes high-level platform KPIs, cross-society membership rolls, circulating credit totals, and onboarding trends.**
GET	**/owner/applications**	**Owner**	**Review Society Applications**	**Lists all pending community network applications awaiting platform owner evaluation.**

PATCH	**/owner/applications/{id}/review**	**Owner**	**Process Application**	**Approves or rejects a society application, automatically provisioning unique invite keys and initializing capacity households.**
GET	**/owner/societies**	**Owner**	**Societies Catalog**	**Fetches the complete catalog of active and federated societies with performance index scores and capacity counts.**
PATCH	**/owner/societies/{id}/suspend**	**Owner**	**Suspend/Reactivate Network**	**Toggles the operational status (is_active) of a specified community network.**

PATCH	**/owner/societies/{id}/reactivate**	**Owner**	**Explicitly Reactivate Network**	**Restores an inactive or suspended society network back to online operational status.**
DELETE	**/owner/societies/{id}/purge**	**Owner**	**Permanently Purge Society**	**Completely deletes a society and its household registry, enforcing safety checks against active resident links.**
GET	**/owner/settings**	**Owner**	**System Manager Settings**	**Retrieves global protocol settings, cluster database status, and manager profile information.**

PATCH	/owner/settings	Owner	Update Manager Profile	Updates the system manager's administrative profile and notification details.

3. Test Cases Table Entry (test_owner.py)

Add these rows to section 4.12 of your project report table:

API Tested	Inputs	Expected Output	Actual Output	Result
GET /owner/dashboard	System owner session	HTTP 200 OK, returns ecosystem-wide activity metrics and pending application queues	HTTP 200, owner metrics and applications returned	PASS
GET /owner/analytics	System owner session	HTTP 200 OK, returns cross-society KPIs, circulating wallet volume, and growth trends	HTTP 200, platform analytics returned	PASS

PATCH /owner/applications/{id}/review	Application ID and approval action payload with max capacity	HTTP 200 OK, approves application, provisions invite key, and initializes capacity households	HTTP 200, application approved successfully	PASS
GET /owner/societies	System owner session	HTTP 200 OK, returns comprehensive catalog of active and federated society networks	HTTP 200, societies catalog returned	PASS
PATCH /owner/societies/{id}/suspend	Valid society ID	HTTP 200 OK, toggles operational status (is_active) to suspend or reactivate network	HTTP 200, network suspension status updated	PASS

4.13 Full Platform Integration Test (test_integration.py)

Workflow Tested	Components Involved	Expected Outcome	Actual Outcome	Result

End-to-End Cross-Module Lifecycle	**Society App (/owner), Resident & Household Allocation (/societies), Wallet Balances & Ledger (/wallet), Marketplace Offerings (/marketplace), and Coordinator Moderation (/coordinator)**	**HTTP 200/201 across modules; successfully provisions a society network, registers resident accounts with initial wallet funding, transitions a skill listing from PENDING to coordinator-approved, and executes an atomic credit transfer (deducting buyer balance, crediting provider balance, and logging dual ledger transactions).**	**Full cross-module workflow executed successfully; balances and ledger entries verified.**	**PASS**